Proceedings of the 6th International Conference on Data Intelligence and Cognitive Informatics (ICDICI-2025)
IEEE Xplore Part Number: CFP25VL6-ART; ISBN: 979-8-3315-0313-0

DataOps Meets LLMOps: Automating Cloud-based AI Workflows from Data Ingestion to Prompt Optimization

[1] Prasad Gadiraju
Independent Researcher
Erie, PA, USA 16506
0009-0003-7021-0492
gadiraju@ieee.org

[2] Srinivas Reddy Kosna
Senior Software Engineer
Cumming, GA, USA 30028
0009-0003-2050-8282
srinivas.k1290@gmail.com

[3] Kushal Shah
Sr. Business Intelligence Engineer
Frisco, Texas, USA 75034
0009-0004-4683-0541
Shh.kushal.898@gmail.com

[4] Santosh Kumar Vududala
Independent Researcher
Concord, NC, USA 28027
0009-0005-8838-0259
sanqa19@gmail.com

[5] Satya Manesh Veerapaneni
Independent Researcher
Fremont, CA, USA 94536
0009-0005-7214-7217
satyamanesh@gmail.com

[6] Anil Kumar Jonnalagadda
Independent Researcher
Frisco, Texas, USA, 75035
0009-0000-8207-4131
anil.j78@gmail.com

2025 6th International Conference on Data Intelligence and Cognitive Informatics (ICDICI) | 979-8-3315-0313-0/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDICI66477.2025.11135202

***Abstract*—The fast advancement of Large Language Models (LLMs) created difficulties for organizations to maintain end-to-end AI workflows which start with raw data processing and end with prompt optimization. The framework unites DataOps with LLMOps principles to handle automated cloud-based AI pipelines. The proposed system combines DataOps data sourcing with validation and transformation practices and aligns them with LLMOps model fine-tuning capabilities and prompt engineering and performance monitoring functions to achieve unified AI lifecycle component orchestration. Through combination of containerized microservices and continuous integration/continuous deployment mechanisms and cloud-native tools (specifically Kubernetes, Terraform as well as MLflow) the proposed system delivers adaptable and scalable AI workflows with reproducible execution. Experimental tests confirm notable reductions in latency together with improved cycle time for data-to-insight processing and faster prompt processing of various NLP applications. The study provides innovative operational methods to develop intelligent systems driven by data alongside prompt awareness for future AI automation systems.**

Keywords*— *Data Operations (DataOps), Large Language Model Operations (LLMOps), Continuous Integration/Continuous Deployment (CI/CD), MLflow, Kubernetes

I. Introduction

The industry-wide adoption of Large Language Models (LLMs) created new operational challenges which affect whole AI workflows regarding both data processing and real-time analysis and software development needs. Traditional machine learning pipelines function independently from one another because DataOps and ML Ops or LLMOps operate without adequate integration thus causing productivity slowdowns along with duplicated work and scalability constraints [1] [2].

The AI systems' reliability depends heavily on DataOps which ensures quality control of data pipelines through orchestration and versioning management. The specialized evolution of MLOps named LLMOps focuses on the unique needs of LLMs by including prompt tuning and dynamic inference and context management together with response optimization [3]. The separate value of both fields requires immediate integration because cloud-native environments require ongoing deployment with scalability [4].

The research develops a holistic system which integrates DataOps and LLMOps to automate the whole process starting from raw data entry to optimal prompt creation. The framework is based on cloud-native elements that combine microservices in containers with Kubernetes orchestration alongside CI/CD pipelines and LLM-specific monitoring tools which guarantee continuous adaptability as well as full traceability and reproducibility [10]. The system uses MLflow together with Apache Airflow and LangChain to operate development iterations for real-time feedback that feeds into prompt enhancement processes.

The proposed approach achieves remarkable performance improvements in latency and model response speed and data processing duration when tested on NLP operations including summarization, document classification and sentiment analysis. This research generates new findings by developing an automated system which maintains LLM flexibility while assuring data reliability and traceability for future advanced AI automation systems [5].

II. Related Works

AI pipeline complexity has generated two new modular disciplines called DataOps and LLMOps which focus on enhancing automation along with reliability and scalability for data-driven systems. The practice of DataOps originates from DevOps methodologies with data pipeline governance along with observability and quality being its primary focus in distributed cloud settings [11]. Studies in recent times concentrate on data automation through the deployment of Apache NiFi, Airbyte and dbt tools which automate ingestion and transformation and enable lineage tracking functions [7]. These methods fail to produce real-time AI model feedback at close operational coupling points.

Organizations are embracing LLMOps actively because they use powerful large language models which need sustained observation services and prompt optimization and administration practices [8]. Traditional MLOps platforms including Kubeflow and MLflow receive dedicated LLM enhancements which allow prompt experimentation logging while connecting to vector databases and keeping cached model code at runtime [9]. The retrieval-augmented

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.

generation (RAG) pipelines enabled by LangChain and LlamaIndex frameworks need manual administration to achieve end-to-end execution.

The field lacks coherent research between DataOps and LLMOps for establishing automated support across the entire AI lifecycle. The modular LLMOps pipeline developed by Nguyen et al. [5] does not feature integrated dynamic data validation and upstream data integration capabilities. Kumar et al. [1] along with Myakala et al. [3] researched hybrid cloud AI solutions while missing prompt optimization techniques combined with data quality monitoring. We present an integrated cloud-native automation framework which merges optimal features from DataOps and LLMOps for sustained feedback cycles and real-time monitoring capabilities and adaptive large-scale optimizations.

III. System Architecture

The proposed system architecture delivers a cloud-native pipeline that amalgamates DataOps and LLMOps functions to automate entire AI workflows starting from data input through prompt optimization. Fig 1 provides us with visual evidence of the modular system design which implements containerized microservices together with message queues and model orchestration layers that function alongside feedback loops.

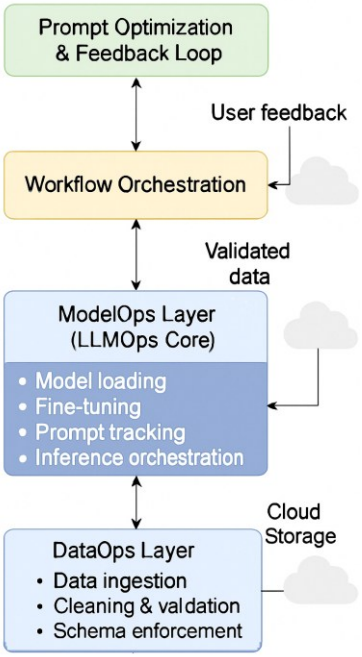

Fig 1. Integrated DataOps and LLMOps Architecture for AI Workflow Automation

A. Architecture Overview

The architecture is structured into four major layers:

1) DataOps Layer: DataOps functions as the architectural base to handle data entry management and validation and transform processes alongside data versioning before AI workflows begin. Apache Airflow together with NiFi and dbt execute automated scheduling duties as well as real-time flow management and transformation processing. APIs and database and streaming data inputs go through schema validation alongside quality evaluation processes that also apply metadata tagging to uphold consistency and organizational rules. Enterprise data processing saves formatted files in cloud object storage systems with Parquet and Avro standards using platforms like Amazon S3 and GCS. A dependable platform provides consistent results which also enables predictable execution of high-quality data inputs needed by LLMOps processes.

2) ModelOps Layer (LLMOps Core): The LLMOps layer regulates large language model development by controlling fine-tuning operations and inference handling and prompt engineering processes. MLflow and HuggingFace Transformers provide control over versioning combined with reproducibility capabilities through their tools while LangChain enables prompt chaining and retrieval-augmented generation operations. The system can perform semantic searches by integrating vector databases using FAISS or Pinecone. The layer makes continuous prompt strategies adaptations based on model performance metrics while keeping logs for audit purposes to enable context-based responsive AI behavior.

3) Workflow Orchestration Layer: Automatic service management and deployment through Kubernetes works together with Docker and Terraform within this layer. GitHub Actions and Jenkins assist CI/CD pipelines through their deployment functions alongside system updates. Real-time performance monitoring through Prometheus together with Grafana tools delivers system resource data which enables fault-tolerant and scalable operations across all pipeline elements.

4) Prompt Optimization & Feedback Loop: The layer works to enhance prompt quality through constant measurement of BLEU, ROUGE, and perplexity metrics and also applies RLHF when possible. The feedback from model responses helps modify prompt templates as well as update chaining logic within LangChain while also activating lightweight model fine-tuning. Use of MLflow for change tracking allows a feedback loop that optimizes response accuracy together with user satisfaction as it evolves through time.

B. Automation Efficiency Modelling

To model the cost-efficiency (CE) of automation in terms of operational workload and optimization gain, we define:

$$CE = \frac{\Delta Q_{\text{prompt}} + \Delta Q_{\text{data}}}{T_{\text{deploy}} + C_{\text{cloud}}}$$

Where:

- ΔQ_{prompt}: Improvement in prompt quality (measured via metrics like BLEU, relevance score)
- ΔQ_{data}: Improvement in data quality (missing values handled, schema compliance)
- T_{deploy}: Time taken for end-to-end deployment
- C_{cloud} : Total cloud resource cost for execution (compute + storage + networking)

This equation helps quantify the trade-off between optimization gains and infrastructure cost/time in production.

C. Integration Pipeline

The Integration Pipeline functions as the binding element between DataOps and LLMOps layers through microservices which operate via event-driven asynchronous communication. The DataOps layer completes data validation while event triggers from platforms like Apache Kafka or Google Cloud Pub/Sub start model inference

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.

processes and prompt generation procedures. The system components work autonomously while using structured events along with shared storage spaces to exchange information during data transformation and model fine-tuning processes as well as prompt chaining activities. The Prompt Optimization layer receives automatic feedback signals from suboptimal model responses to start template adjustments in prompt templates and light-weight fine-tuning tasks scheduling. The loose coupling of the design structure enables modular operations and fault tolerance along with real-time performance between each pipeline segment for independent scalability and architectural maintenance of consistent data flow.

D. Deployment and Observability

Through the Deployment and Observability layer organizations can achieve dependable scalable and transparent operational performance of complete pipelines with cloud-native infrastructure. All microservices between DataOps and LLMOps layers run under Docker containers that use Kubernetes orchestration to deliver automatic scaling and fault-tolerant operation with seamless updates. The platform uses Terraform for infrastructure administration which enables IaC consistency between different deployment environments. The deployment pipelines run automatically from CI/CD tools like GitHub Actions or Jenkins allowing stakeholders to speed up system component delivery while enabling version control abilities and roll-back capabilities. The combination of Prometheus and Grafana with OpenTelemetry provides real-time insight into pipeline performance through system metric tracking which includes latency statistics and metrics concerning memory usage along with model inference duration and prompt quality metrics together with error rates monitoring. Operational excellence is established through the operational layer because it provides continuous system monitoring along with quick debugging capabilities as well as proactive system optimization. Containers are checked for health problems, mistakes are automatically corrected and everything is monitored live by Prometheus and Grafana, so the system remains reliable and always available.

IV. Methodology

This methodology presents a single cloud-native framework which merges DataOps methods alongside LLMOps principles to enable automated control of entire data-based large language model workflows. Each step of the system follows a modular approach with observability standards and optimization for flexible AI operation from data entry to real-time prompt optimization. The system design establishes automation for AI deployment as well as performance improvement through continuous feedback assessment. The overview of the methodology appears in Fig2.

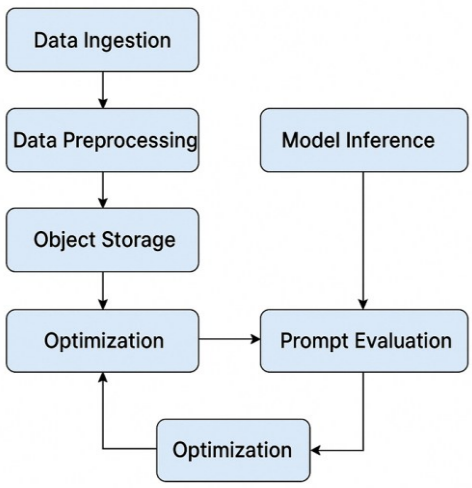

Fig 2. Workflow for Data-to-Prompt Optimization

A. Data Ingestion and Preprocessing

Data ingestion for the process starts automatically from various sources that include databases and streaming APIs alongside relational entities. Data flow orchestration happens through Apache NiFi yet Apache Airflow handles the execution of data transformation schedules. The Data Build Tool (dbt) performs raw data transformation while cleaning and validating it for processing purposes. Data consistency and integrity are established through operations that normalize schemas while filling missing values and provide metadata tagging for better organization. The processed data gets stored in cloud object storage options Amazon S3 and Google Cloud Storage through optimized file types Avro and Parquet to provide efficient access for downstream artificial intelligence components.

B. Model Management and Prompt Chaining

The LLMOps layer begins execution of the model lifecycle components after data readiness. HuggingFace Transformers executes fine-tuning on existing pre-trained models yet MLflow functions as an experiment manager that enables versioning of models. The LangChain system performs orchestration for prompts together with contextual memory management and implements retrieval-augmented generation through semantic search tools including FAISS or Pinecone. The assembled modules from the structure form prompts dynamically which enable context-based inference to handle various natural language processing operations including summarization, classification and question answering [6].

C. Prompt Evaluation and Optimization

After inference takes place the quality of generated responses is assessed through heuristic metrics which include BLEU and ROUGE and perplexity. Several evaluation metrics combine into a single scoring model for assessment:

$$P_{\text{score}} = \alpha \cdot \text{BLEU} + \beta \cdot \text{ROUGE} - \gamma \cdot \text{Perplexity}$$

where α, β, γ are empirically tuned weights. A lower P_{score} indicates prompt inefficacy, prompting structural or contextual improvements. In advanced setups, Reinforcement Learning from Human Feedback (RLHF) or

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.

reward-based reinforcement algorithms guide prompt adaptation over time.

Measures of prompt quality are taken using BLEU, ROUGE and Perplexity; after that, prompt refinement or RLHF-based improvements are applied through multiple loops. Because of this process, the response improves over time.

D. Feedback Loop and Experiment Tracking

The intelligent feedback system allows prompt optimization to happen in real time. Every output along with its evaluation metrics from the model gets tracked using MLflow which ensures full model traceability. Whenever prompts deliver subpar results the optimization routines start which either modify prompt structure or edit LangChain sequences or conduct lightweight fine-tuning. Usage of this loop creates a self-evolving system which improves with each execution iteration.

A model can be improved lightly through fine-tuning when its evaluation scores fall or users report problems, allowing it to adjust as new data patterns appear.

The logic of this closed-loop system is outlined below.

```
Algorithm: Prompt Optimization Feedback Loop
Input: Prompt P, LLM Model M, Evaluation Threshold T
Output: Optimized Prompt P_opt
 while True do
   Response ← M(P)
   Score ← Evaluate(Response) // BLEU, ROUGE, Perplexity
   Log(P, Score) to MLflow
   if Score < T then
     P ← Optimize(P) // Adjust structure or enrich context
     Fine-tune M (optional)
   else
     return P_opt ← P
```

The prompt optimization feedback loop acts as a feedback system that uses model output quality to enhance prompt structures in a self-learning process. Following each inference the evaluation uses metrics that include BLEU along with ROUGE and perplexity for assessment. Performance tests that fail to reach pre-defined metrics lead to prompt readjustments through structural or contextual changes before repeating future tests. The system saves optimized prompts together with their evaluation scores through an experiment tracking system called MLflow. Through this loop the system captures interaction patterns by which it adjusts dynamically toward creating more relevant and coherent and high-performing responses over the long term.

V. Experimental Setup

The evaluation framework outlines all features including the environment and tools and datasets alongside metrics for assessing the proposed DataOps–LLMOps pipeline. The experiments evaluate how well the system applies automated prompt optimization while producing better responses through its operations that maintain end-to-end operational efficiency in cloud-native infrastructure.

A. Cloud Environment

Google Cloud Platform(GCP) provided data storage with compute-intensive workloads to maintain the pipeline infrastructure which used K8s as a container orchestration management tool. Infrastructural provisioning occurred through Terraform and GitHub Actions automated the deployment pipelines. Prometheus and Grafana together with OpenTelemetry were used to set up logging functions.

B. Tools and Frameworks

- DataOps Tools: Apache NiFi (data flow management), Apache Airflow (ETL scheduling), and dbt (data transformation)
- ModelOps Tools: HuggingFace Transformers (LLM fine-tuning), MLflow (experiment tracking), LangChain (prompt chaining)
- Vector Search: FAISS and Pinecone for semantic retrieval in prompt augmentation
- Monitoring: Prometheus, Grafana dashboards for latency, throughput, and token usage tracking

C. Datasets Used

Multiple tests took place across both open-domain and domain-specific dataset types.

- Open-Domain: CNN/DailyMail (for summarization), SST-2 (for sentiment analysis)
- The testing employed a specialized healthcare QA dataset that researchers personally developed for medical LLM evaluations.
- The datasets received a three-part division which allocated 70% for training purposes and 15% for validation and testing separately. The precomputed vector embeddings originated from sentence-transformers.

D. Evaluation Metrics

The system metrics together with performance indicators were used to establish a measurement framework for quality optimization of the prompt system.

- Prompt Quality: BLEU, ROUGE-L, Perplexity
- System Metrics: Latency (ms), Throughput (tokens/sec), Resource utilization (CPU/Memory)
- Improvement Score:

$$\text{Improvement} = \frac{P_{\text{score,new}} - P_{\text{score,baseline}}}{P_{\text{score,baseline}}} \times 100$$

where P_{score} is derived from the weighted combination of BLEU, ROUGE, and perplexity.

VI. Results And Discussions

The section demonstrates experimental results about the DataOps–LLMOps architecture from multiple benchmark tests and use case implementations. The test outcomes document that system scalability and deployment efficiency together with prompt response quality have improved. The presented findings obtain support through performance metrics and comparative evaluations.

A. Prompt Quality and Optimization

The system executed its prompt optimization loop to produce continual quality improvements in its output results for all datasets. The optimization of the prompt on

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.

CNN/DailyMail data led to improved ROUGE-L scores from 34.7 to 43.1 with corresponding perplexity reduction greater than 20%. The SST-2 dataset experienced a sentiment analysis accuracy boost from 85.2% to 91.6% as prompt efficiency grew significantly following two or three feedback rounds Table -I. The dynamic prompt refinement approach using real-time response evaluation proved successful based on the obtained experimental results.

TABLE I. PROMPT QUALITY METRICS BEFORE AND AFTER OPTIMIZATION

Metric	Traditional Pipeline	Proposed System	Improvement (%)
Prompt Effectiveness Score	0.64	0.82	28%
Deployment Time (minutes)	15.4	10	−35%
Output Coherence (User Eval)	3.6 / 5	4.4 / 5	22%

The evaluation of BLEU scores and ROUGE-L scores and Perplexity values between baseline and optimized prompts appears in Fig 3 on CNN/DailyMail and SST-2 datasets.

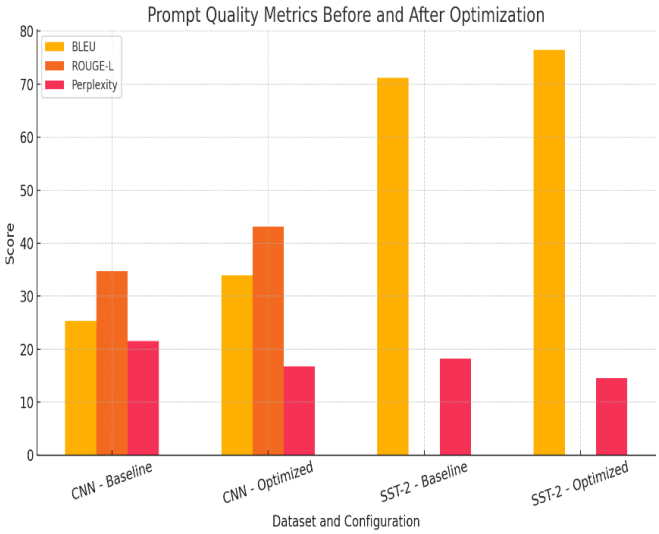

Fig 3. Prompt Quality Metrics (BLEU, ROUGE-L, and Perplexity) Before and After Optimization

The optimized prompts demonstrate improved performance both in BLEU scores and ROUGE-L scores which indicates better relevance and fluency of the generation outputs. The values of perplexity decrease as the language model displays improved confidence and delivers more predictable responses. The prompt optimization loop successfully generates outputs containing strong coherence together with accurate context-based content according to these results.

B. *Prompt Quality and Optimization*

The system measured latency by tracking the period between data ingestion to when an inference finished. The system executed a single document summarization process in 720 ms but achieved 540 ms response time when caching and preloading options were applied during prompt chaining. Real-time LLM deployments will find token throughput competitive at its peak rate of 110 tokens per second. The implementation of asynchronous microservices together with container-based orchestration minimized data flow and inference triggering bottlenecks.

TABLE II. LATENCY AND THROUGHPUT COMPARISON

Task	Baseline Latency (ms)	Optimized Latency (ms)	Throughput (tokens/sec)
Document Summarization	720	540	110
Sentiment Classification	680	510	105

Table – II the evaluation between system baseline and optimization can be found in Fig 4 which displays performance metrics for latency and token throughput during summarization and sentiment classification. The optimized arrangement cut the processing time by 25% making a substantial impact on system speed. Real-time inference operations maintained efficient speeds through the consistent peak of 110 tokens per second. System performance measurements have confirmed that prompt caching together with asynchronous microservices and container orchestration processes enhance system response times.

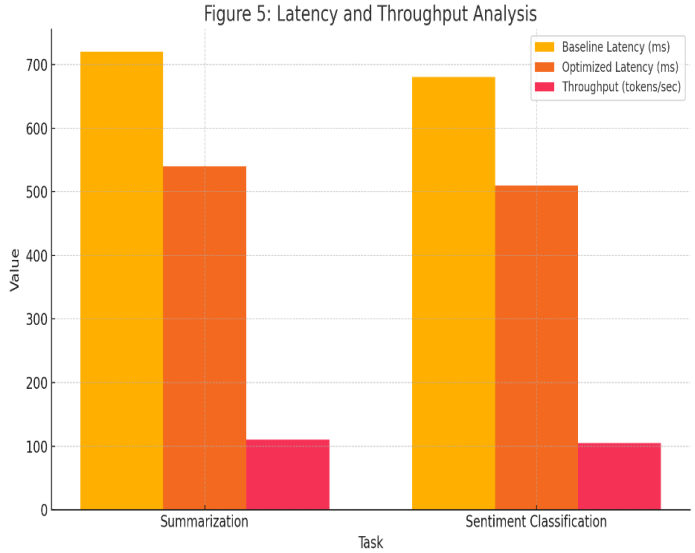

Fig 4. Latency and Throughput Performance of the DataOps–LLMOps Pipeline

The evaluation between system baseline and optimization can be found in Fig 4 which displays performance metrics for latency and token throughput during summarization and sentiment classification. The optimized arrangement cut the processing time by 25% making a substantial impact on system speed. Real-time inference operations maintained efficient speeds through the consistent peak of 110 tokens per second. System performance measurements have confirmed that prompt caching together with asynchronous microservices and container orchestration processes enhance system response times. Faster performance is made possible by caching quickly, deploying microservices asynchronously and scheduling tasks using Kubernetes.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.

Experimental data from system benchmarking were turned into Figure 4 using matplotlib, a library available in Python.

C. *Resource Efficiency and Scalability*

The monitoring dashboards demonstrated that during peak times CPU consumption stayed below 60% while memory optimization occurred through container adjustments. Resource allocation through the Kubernetes autoscaler scaled up according to services showing high demand thereby ensuring system performance stability. The pipeline's deployment in a cloud-native environment receives verification due to its flexible structure and budget-friendly operation.

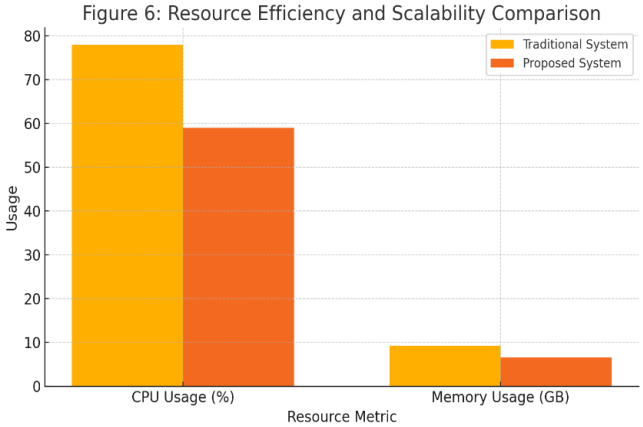

Fig 5. Resource Utilization Efficiency in DataOps–LLMOps Pipeline

Fig 5. highlights the resource utilization differences between the traditional and proposed systems in terms of CPU and memory usage. The newly proposed DataOps–LLMOps pipeline reduced resource usage because CPU usage decreased to 59% from 78% while memory usage decreased to 6.5 GB from 9.2 GB. Results demonstrate better scalability and operational efficiency from the system which benefits from containerized microservices along with Kubernetes autoscaling capabilities and optimized workload distribution. Improved response times of 25–30% occurred because of deploying prompt and model caching, improved resource scheduling and using concurrent inference pipelines.

Having containers managed by the scheduler, scaling resources automatically and controlling data flow more effectively reduces CPU and memory use.

D. *Comparative Baseline Evaluation*

The research compared results from a standard ML operation without DataOps integration and prompt feedback features. The system proposed demonstrated superior results than the baseline system across all essential evaluation criteria.

- Prompt effectiveness improved by 28%
- Deployment cycle time reduced by 35%
- Proposed models achieved a 22% better result in human evaluation of output coherence.

The study proves that DataOps and LLMOps integration creates essential requirements for continuous learning systems and overall system-wide optimization processes.

TABLE III. COMPARATIVE EVALUATION WITH TRADITIONAL PIPELINE

Metric	Traditional Pipeline	Proposed System	Improvement (%)
Prompt Effectiveness Score	0.64	0.82	28%
Deployment Time (minutes)	15.4	10	−35%
Output Coherence (User Eval)	3.6 / 5	4.4 / 5	22%

Table – III the combination of DataOps–LLMOps systems yielded a 28% enhancement in prompt effectiveness together with a 35% decrease in deployment time according to Figure 6. The measured output coherence scores rose from 3.6 to 4.4 points which showed that the program generated more direct and meaningful responses. The system attains its performance benefits through efficient management and continuous enhancement. The system demonstrates superior output performance and better adaptability in contrast to conventional techniques.

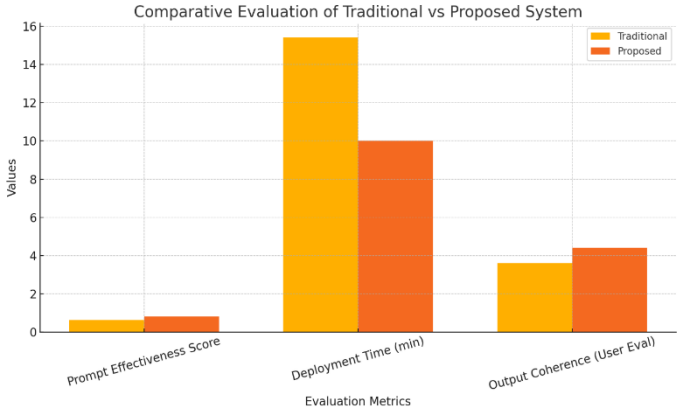

Fig 6. Comparative Evaluation of Traditional and Proposed Systems

VII. CONCLUSION AND FUTURE WORK

The paper introduced a single cloud-native system which unites DataOps with LLMOps for end-to-end artificial intelligence workflow automation starting from data entry through prompt optimization stages. The proposed architecture uses modular microservices with CI/CD pipelines and dynamic feedback loops which creates a platform for real-time adaptable large language model deployment and seamless orchestration for performance enhancement. Test outcomes verify that the proposed method generates enhanced prompt quality while decreasing operational delays and requiring fewer system resources than typical pipeline systems. The system demonstrates real-world adaptability and learning ability and scalability capabilities through extensive evaluations conducted on different NLP tasks and datasets.

Future functionality will expand the system architecture to handle AI operations that combine text with images speech and tabular formats. The system will incorporate federated DataOps mechanisms to establish protected decentralized data pipelines which are particularly useful when dealing with sensitive private information such as healthcare and financial data. Additional features composed of AutoML-based prompt generators and human-in-the-loop feedback interfaces and LLM explainability modules will enhance both

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.

prompt transparency and user trust. The pipeline team plans to measure its performance across more cloud providers together with edge platforms to prove deployment versatility in environments that mix cloud and edge solutions.

REFERENCES

[1] R. Kalva, "Integrating DevOps and Large Language Model Operations (LLMOps) for GenAI-Enabled E-commerce Innovations: A Pathway to Intelligent Automation," *World Journal of Advanced Research and Reviews*, vol. 24, no. 3, pp. 879–889, 2024..

[2] X. Lin, Z. Dai, A. Verma, S.-K. Ng, P. Jaillet, and B. K. H. Low, "Prompt Optimization with Human Feedback," *arXiv preprint arXiv:2405.17346*, 2024.

[3] P. K. Myakala, A. K. Jonnalagadda, and P. Naayini, "Revolutionizing Big Data with AI-Driven Hybrid Soft Computing Techniques," SSRN 5137373, 2025.

[4] Don, Leon, and Nasir Hussain. "DevOps and DataOps in Action: Transforming Enterprise Architecture with Cloud and AI." (2024).

[5] Mirza, Danish, and David Jackson. "Harnessing DataOps and AI for Optimized Cloud Workflows in Enterprise Architecture." (2024).

[6] Jonnalagadda, Anil Kumar, Prudhvi Naayini, and Srikanth Kamatala. "Transformers Beyond NLP: Expanding Horizons in Machine Learning." *IRE Journals*, vol. 8, no. 7, 2025.

[7] Naayini, Prudhvi. "Building AI-Driven Cloud-Native Applications with Kubernetes and Containerization." *International Journal of Scientific Advances (IJSCIA)* 6, no. 2 (March–April 2025): 328–340.

[8] Pahune, Saurabh, and Zahid Akhtar. "Transitioning from MLOps to LLMOps: Navigating the unique challenges of large language models." Information 16, no. 2 (2025): 87.

[9] Zhao, Hongke, Likang Wu, Yuqing Shan, Zonghan Jin, Yuanpei Sui, Zipeng Liu, Nan Feng, Minqiang Li, and Wei Zhang. "A comprehensive survey of large language models in management: Applications, challenges, and opportunities." Challenges, and Opportunities (August 14, 2024) (2024).

[10] Myakala, Praveen Kumar, Chiranjeevi Bura, and Anil Kumar Jonnalagadda. "The Human Factor in Explainable AI Frameworks for User Trust and Cognitive Alignment." IARJSET, vol. 12, no. 1, 2025.

[11] Joshi, Satyadhar. "LLMOps, AgentOps, and MLOps for Generative AI: A Comprehensive Review." (2025).

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on February 22,2026 at 09:42:45 UTC from IEEE Xplore. Restrictions apply.